

PRACTICAL - 9

Aim: 1. Counter race: several threads each increment a shared counter many times. Run it without synchronization to watch the total come out too low, then make the increment synchronized and confirm it is exactly right every run.

Code Screenshot:

CounterRace.java:-

Program 1A - Without Synchronization

```

1  class Counter {
2      int count = 0;
3
4      void increment() {
5          count++;
6      }
7  }
8
9  class CounterThread extends Thread {
10     Counter counter;
11     int times;
12
13     CounterThread(Counter counter, int times) {
14         this.counter = counter;
15         this.times = times;
16     }
17
18     public void run() {
19         for (int i = 0; i < times; i++) {
20             counter.increment();
21         }
22     }
23 }
24
25 public class CounterRace {
26     Run | Debug
27     public static void main(String[] args) throws InterruptedException {
28
29         Counter counter = new Counter();
30
31         int numberOfThreads = 10;
32         int incrementsPerThread = 100000;
33
34         Thread[] threads = new Thread[numberOfThreads];
35
36         for (int i = 0; i < numberOfThreads; i++) {
37             threads[i] = new CounterThread(counter, incrementsPerThread);
38
39             threads[i].start();
40         }
41
42         for (int i = 0; i < numberOfThreads; i++) {
43             threads[i].join();
44         }
45
46         int expected = numberOfThreads * incrementsPerThread;
47
48         System.out.println("Expected count: " + expected);
49         System.out.println("Actual count: " + counter.count);
50     }
51 }

```

Output Screenshot:

```
PS C:\Users\BHAHTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> javac CounterRace.java
PS C:\Users\BHAHTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> java CounterRace
Expected count: 1000000
Actual count: 705384
PS C:\Users\BHAHTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> █
```

Program 1B - With Synchronization

```
1  class Counter {
2      int count = 0;
3
4      void increment() {
5          count++;
6      }
7  }
8
9  class CounterThread extends Thread {
10     Counter counter;
11     int times;
12
13     CounterThread(Counter counter, int times) {
14         this.counter = counter;
15         this.times = times;
16     }
17
18     public void run() {
19         for (int i = 0; i < times; i++) {
20             counter.increment();
21         }
22     }
23 }
24
25 public class CounterRace_1 {
26     Run | Debug
27     public static void main(String[] args) throws InterruptedException {
28         Counter counter = new Counter();
29
30         int numberOfThreads = 10;
31         int incrementsPerThread = 100000;
32
33         Thread[] threads = new Thread[numberOfThreads];
34
35         for (int i = 0; i < numberOfThreads; i++) {
36             threads[i] = new CounterThread(counter, incrementsPerThread);
37             threads[i].start();
38         }
39
40         // Wait for all threads
41         for (int i = 0; i < numberOfThreads; i++) {
42             threads[i].join();
43         }
44
45         int expected = numberOfThreads * incrementsPerThread;
46
47         System.out.println("Expected count: " + expected);
48         System.out.println("Actual count: " + counter.count);
49     }
50 }
```

Output Screenshot:

```
PS C:\Users\BHAHTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> javac CounterRace_1.java
PS C:\Users\BHAHTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> java CounterRace_1
Expected count: 1000000
Actual count: 424874
PS C:\Users\BHAHTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> █
```

Aim : 2 Seat booking race: a shared seatsLeft of 5 with about 10 threads each trying to book one seat. Without synchronization show that more than 5 can succeed (oversell); with a synchronized book() exactly 5 do.

2A. Without Synchronization

SeatBookingRace.java :-

```

1  class SeatBooking {
2      int seatsLeft = 5;
3      boolean book() {
4          if (seatsLeft > 0) {
5              try {
6                  Thread.sleep(millis: 10);
7              } catch (InterruptedException e) {
8                  Thread.currentThread().interrupt();
9              }
10             seatsLeft--;
11             return true;
12         }
13         return false;
14     }
15 }
16 class BookingThread extends Thread {
17     SeatBooking booking;
18     BookingThread(SeatBooking booking) {
19         this.booking = booking;
20     }
21     public void run() {
22         boolean success = booking.book();
23         if (success) {
24             System.out.println(
25                 Thread.currentThread().getName() +
26                 " booked a seat"
27             );
28         } else {
29             System.out.println(
30                 Thread.currentThread().getName() +
31                 " failed - no seat"
32             );
33         }
34     }
35 }
36
37 public class SeatBookingRace {
38
39     Run | Debug
40     public static void main(String[] args) throws InterruptedException {
41         SeatBooking booking = new SeatBooking();
42
43         int numberOfThreads = 10;
44
45         Thread[] threads = new Thread[numberOfThreads];
46
47         for (int i = 0; i < numberOfThreads; i++) {
48             threads[i] = new BookingThread(booking);
49             threads[i].setName("Customer-" + (i + 1));
50
51             threads[i].start();
52         }
53
54         for (int i = 0; i < numberOfThreads; i++) {
55             threads[i].join();
56         }
57
58         System.out.println("Seats left: " + booking.seatsLeft);
59     }
60 }
61 }

```

Output Screenshot:

```

PS C:\Users\BHAHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> javac SeatBookingRace.java
PS C:\Users\BHAHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> java SeatBookingRace
Customer-3 booked a seat
Customer-2 booked a seat
Customer-6 booked a seat
Customer-1 booked a seat
Customer-10 booked a seat
Customer-5 booked a seat
Customer-9 booked a seat
Customer-8 booked a seat
Customer-4 booked a seat
Customer-7 booked a seat
Seats left: -5

```

2B. With Synchronization :

```

1  class SeatBooking {
2      int seatsLeft = 5;
3      synchronized boolean book() {
4          if (seatsLeft > 0) {
5              try {
6                  Thread.sleep(millis: 10);
7              } catch (InterruptedException e) {
8                  Thread.currentThread().interrupt();
9              }
10             seatsLeft--;
11             return true;
12         }
13         return false;
14     }
15 }
16 class BookingThread extends Thread {
17     SeatBooking booking;
18     BookingThread(SeatBooking booking) {
19         this.booking = booking;
20     }
21     public void run() {
22         boolean success = booking.book();
23         if (success) {
24             System.out.println(
25                 Thread.currentThread().getName() +
26                 " booked a seat"
27             );
28         } else {
29             System.out.println(
30                 Thread.currentThread().getName() +
31                 " failed - no seat"
32             );
33         }
34     }
35 }
36
37 public class SeatBookingRace_1{
38
39     Run | Debug
40     public static void main(String[] args) throws InterruptedException {
41
42         SeatBooking booking = new SeatBooking();
43
44         int numberOfThreads = 10;
45
46         Thread[] threads = new Thread[numberOfThreads];
47
48         for (int i = 0; i < numberOfThreads; i++) {

```

```

49         threads[i] = new BookingThread(booking);
50         threads[i].setName("Customer-" + (i + 1));
51
52         threads[i].start();
53     }
54
55     for (int i = 0; i < numberOfThreads; i++) {
56         threads[i].join();
57     }
58
59     System.out.println("Seats left: " + booking.seatsLeft);
60 }
61 }
62

```

Output Screenshot:

```

PS C:\Users\BHAkti PATEL\OneDrive\Desktop\CHARUSAT\OOPS> javac SeatBookingRace_1.java
PS C:\Users\BHAkti PATEL\OneDrive\Desktop\CHARUSAT\OOPS> java SeatBookingRace_1
Customer-2 booked a seat
Customer-7 booked a seat
Customer-9 booked a seat
Customer-8 booked a seat
Customer-10 booked a seat
Customer-5 failed - no seat
Customer-6 failed - no seat
Customer-4 failed - no seat
Customer-1 failed - no seat
Customer-3 failed - no seat
Seats left: 0
PS C:\Users\BHAkti PATEL\OneDrive\Desktop\CHARUSAT\OOPS>

```

Learning Outcome:

The three programs demonstrated the importance of thread synchronization when multiple threads access shared data. In the Counter Race program, unsynchronized access could produce an incorrect counter value, while using synchronized ensured the expected result every time. In the Seat Booking Race, lack of synchronization could cause more than the available 5 seats to be booked, whereas a synchronized book() method allowed exactly 5 successful bookings. The Advanced Array Sum program further showed how shared data can cause race conditions and how synchronization or separate partial results can solve the problem. Overall, these experiments demonstrated that synchronization prevents race conditions and maintains data consistency in multithreaded programs, although synchronization may introduce some performance overhead.

